

Multi-Bottleneck Fairness on RDMA Fabrics: An Empirical Study of Deployable Congestion Control and an RCP Service Mode over In-Network Telemetry

Haoyu Wang*, Bo Sheng*, Zerín Shaima Meem†, Xiaoqian Zhang†

*University of Massachusetts Boston, Boston, MA, USA {haoyu.wang001, bo.sheng}@umb.edu

†University of Nebraska Omaha, Omaha, NE, USA {zmeem, xiaoqianzhang}@unomaha.edu

Abstract—Deployable RDMA congestion control is evaluated almost exclusively on single-bottleneck benchmarks, while datacenter traffic increasingly crosses several contended links in series. We measure multi-bottleneck fairness on lossless RDMA fabrics against a fluid max-min bandwidth-allocation bound: every scheme we evaluate (DCQCN, TIMELY, DCTCP, HPCC) is unfair to the multi-bottleneck flow (1.6–2.3×), worsening with bottleneck count; HPCC’s instance is a stable near-winner-take-all collapse to ~ 0.4 Gbps. Six sender-only corrections fail — including a private mirror of the RCP recursion driven by the same INT signals: independently maintained explicit-rate state preserves arrival-history asymmetry, while one shared per-link register erases it — and per-flow fair queueing equalizes only what the endpoint offers. Carrying that register’s value in one 64-bit INT field (RDMA-RCP, three words of state per port) restores max-min — 1.005×, every flow within 5–13% of its oracle FCT in heterogeneous scenarios (21–39% under compound stress), zero PFC — and cuts a ring-collective coflow’s completion time by 18–23% (18% under byte-identical headers). We equally measure where the mode does not belong: it cannot share an unsliced link with stock HPCC, and fan-in past ~ 64 needs a lengthened control interval (25–50% utilization at the default; 84–94% at 2–4×). Stock HPCC’s code remains byte-for-byte unchanged.

Index Terms—datacenter networks, congestion control, RDMA, in-network telemetry, max-min fairness, RCP

I. INTRODUCTION

HPCC [1] and its derivatives represent the state of the art in datacenter RDMA congestion control: the switch stamps raw per-hop state into INT headers, and the sender computes a precise utilization estimate toward a target η , matching or beating DCQCN, TIMELY, and DCTCP on the canonical single-bottleneck benchmarks — which is where deployable RDMA congestion control is, almost exclusively, evaluated.

Modern datacenter traffic, however, increasingly crosses *several contended links in series*: cross-pod ML collectives, disaggregated memory and storage, multi-tenant aggregation and core layers. The practical stake is concrete: a collective completes when its *slowest* flow completes, and the slowest flows are precisely the cross-fabric, multi-bottleneck ones. On a 16-flow ring-collective coflow over a $k=4$ fat-tree, we measure stock HPCC inflating job completion time by $\sim 27\%$ over what max-min allocation delivers — at every ECMP hash seed we test (§V-D). Whether deployable RDMA congestion control

is fair to such traffic has not been carefully measured. This paper measures it.

Max-min fairness is a policy choice, not a universal congestion-control objective; our claims target traffic classes whose completion is gated by multi-bottleneck flows, and §V-N delimits where it is the wrong target. We proceed with a deterministic, oracle-grounded methodology. Our simulator is the original HPCC authors’ ns-3 code base, upgraded to a tagged modern release (ns-3.45): the changes are interface-level — build-system, header, and deprecated-API adaptations — while the congestion-control logic, switch datapath, and wire formats are the original code (one latent field-initialization bug, exposed only by the 376-node topology, is fixed and documented). We validated the upgrade by reproducing the original paper’s baseline comparisons across five schemes and two production workloads (§V); the reproduction record and raw results are public. The measurements here are therefore produced by the original HPCC implementation’s logic, not a re-implementation. Our findings:

- 1) **The failure spans every scheme we evaluate, and we characterize HPCC’s instance (§II, §V-C).** Every deployable scheme we test is unfair to the multi-bottleneck flow — DCQCN 1.70–2.32×, TIMELY 1.83–2.01×, DCTCP 1.61–1.78×, HPCC 1.90–1.96× at $N = 2-4$ — all growing with bottleneck count, and robust to start-time jitter. For HPCC we isolate the mechanism: the failure is *near-winner-take-all* (the long flow collapses to ~ 0.4 Gbps against ~ 23 Gbps competitors; $3.5\times$ for small flows), RTT-independent, immune to HPCC’s own per-hop multi-rate mode and to INT compression (PINT), and compounded past four bottlenecks by a wire-format cap ($\text{maxHop} = 5$) that corrupts the control input. A single-bottleneck control ($N=1$: 1.10–1.39× pairwise spread, direction set by arrival order) separates HPCC’s baseline rate lottery from the systematic multi-bottleneck penalty on top.
- 2) **Private replication of the fair-rate computation cannot close the gap — shared placement can (§III).** We evaluate six sender-only designs: five heuristics, plus the

most direct variant — a full sender-side mirror of the RCP recursion driven by the same INT-visible inputs the switch would use. The mirror still fails ($1.56\times$ even with synchronized starts; $1.65\times/0.81\times$ under staggered arrival, the *direction* set by arrival order), while the identical law at the switch holds $\approx 1.0\times$; per-flow fair queueing equalizes only what senders offer — windowed HPCC leaves $1.97\times$, windowless HPCC equalizes at $\sim 13\times$ the oracle FCT. The mechanism is history: independently maintained rate state preserves arrival-order asymmetry indefinitely, while a shared per-link register eliminates it — evidence about explicit-rate state placement, not a formal impossibility (§III).

- 3) **An RCP service mode over INT restores max-min, verified end to end** (§IV, §V). RDMA-RCP carries a switch-computed RCP fair rate in one 64-bit INT field with three words of mutable state per egress port, alongside byte-for-byte unchanged stock HPCC. It reaches penalty $\approx 1.005\times$ across bottleneck counts, asymmetric sizes, staggered and jittered starts, and ECMP hash seeds — rate-only or with a canonical RCP window — cutting ring-collective coflow JCT by 18–23% (18% under a byte-identical INT header). We equally report its deployment boundary: sharing a link with stock HPCC serves neither class, even under a weighted-DRR reservation, because both controllers read port-global telemetry (§IV-C) — a fabric-wide or isolated-service mode. The mechanism is RCP; we claim the measurement study and the placement result, not a new control law.

Artifacts. The implementation, the topology/workload generators, the max-min oracle, and deterministic scripts that regenerate the figures (`make_figures.py`) and the ablation/sensitivity/baseline tables (`run_sensitivity.py`) from source are available in our artifact-review repository.¹

II. BACKGROUND AND MOTIVATION

A. HPCC, briefly

HPCC’s per-hop utilization estimate for hop i is

$$u_i = \frac{txRate_i}{C_i} + \frac{\min(qlen_i, qlen_i^{prev}) \cdot maxRate}{C_i \cdot W} \quad (1)$$

where the first term reflects bandwidth utilization and the second is a queue-drain term that keeps standing queues bounded. In single-rate mode the sender takes $U = \max_i u_i$, smooths it over the base RTT, and applies

$$new_rate = \begin{cases} curRate / \max c + R_{AI} & \max c \geq 1 \\ curRate + R_{AI} & \text{otherwise} \end{cases}$$

with $\max c = U/\eta$. The window W is sized to $m_{win} \cdot rate/NIC_line$ under `var_win`.

¹<https://github.com/UNOCourseDemo/hpcc-fs>

B. The multi-bottleneck parking lot

The *parking lot* is a classic congestion-control fairness benchmark [2], [3]. Because every link is equally contended, the *max-min fair* allocation [2] gives every flow an equal share, so the long flow should run exactly as fast as each cross flow despite traversing many contention points rather than one. The parking lot therefore isolates the multi-bottleneck fairness question in its simplest form, with no confounding asymmetry in link capacity or hop count of the competitors. One control is essential before reading any deviation as a multi-bottleneck effect: on a *single* shared bottleneck ($N=1$) with two symmetric 50 MB flows, stock HPCC already allocates unequally (37.20 vs 26.72 ms, a $1.39\times$ spread), while RDMA-RCP equalizes them exactly (33.89 vs 33.89 ms). Stock HPCC therefore carries a baseline rate unfairness between symmetric flows; the multi-bottleneck penalty we characterize is the *additional* deviation on top of it, growing to $1.90\text{--}1.96\times$ at $N = 2\text{--}4$.

We construct a parameterized parking lot with N inter-switch bottlenecks (25 Gbps) and 100 Gbps host links (Figure 1): one “long” flow crosses all N , one “cross” flow crosses each; 50 MB each, simultaneous start, so the max-min rate is 12.5 Gbps for every flow. All experiments are deterministic (byte-identical across repeated runs).

Metric. We compare each flow’s actual flow completion time (FCT) against a **fluid max-min bandwidth-allocation lower bound** (“oracle” hereafter) — a continuous model that allocates *payload* bandwidth max-min fairly at every instant, with no propagation, packetization, header, queueing, or feedback cost; live protocols therefore sit strictly above it. A solo calibration flow (no contention) completes 5.8% above the bound on wire and protocol overhead alone (`run_round5.py`), anchoring how much of a small P_i excess is accounting, not control. We compute the allocation by *progressive filling* [2]: all active flows rise in lockstep until a link saturates; those flows freeze; repeat until all are frozen. Because flows complete and depart over time, we run it *event-driven* — recomputing the allocation at each flow completion — so the oracle FCT accounts for bandwidth freed as flows finish; it is exact for our fluid topologies. The headline metric is the **unfairness ratio** $\max_{\text{path}} FCT / \text{mean}(\text{short-path FCT})$; the oracle’s value is 1.0 for equal-size, simultaneous flows.

C. Findings F1–F4: characterizing the gap

Magnitude. Stock HPCC’s unfairness ratio at $N \in \{2, 3, 4\}$ is 1.90, 1.92, 1.96: the multi-bottleneck flow’s FCT is roughly twice that of single-bottleneck competitors. **There is a non-monotonic regime change at $N \geq 5$** , where the long flow becomes favored ($0.51\times$); this turns out to be an INT-overflow artifact at $\max\text{Hop} = 5$ and we exclude $N \geq 5$ from the rest of this paper’s stock comparisons (Figure 2).²

²The 5-hop cap is HPCC’s original wire format, not an artifact of our ns-3 upgrade: the reference simulator hard-codes $\max\text{Hop} = 5$ in its INT header [4], consistent with the paper’s 42-byte worst-case INT overhead (five 8-byte hop records). Conversely, a parking-lot path at $N \leq 4$ traverses at most five switches — *within* the INT budget — so the retained $N = 2\text{--}4$ measurements are structurally free of the overflow.

Parking lot at $N=4$: one LONG flow + one CROSS flow per bottleneck

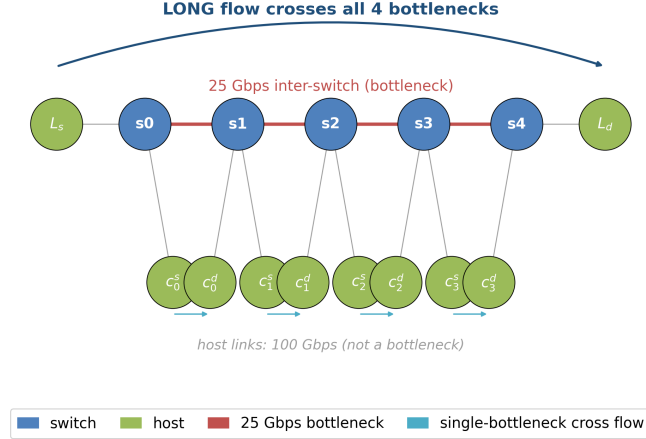


Fig. 1. Parking-lot benchmark at $N = 4$. The long flow $L_s \rightarrow L_d$ crosses every bottleneck; each cross flow $c_i^s \rightarrow c_i^d$ traverses exactly one. Host links are not bottlenecks.

Robustness. Size and start-time perturbations give $1.81\text{--}3.53\times$ (Figure 4), worst ($3.5\times$) when the multi-bottleneck flow is small — the latency-sensitive RPC case. *Multi-rate HPCC* (per-hop EWMA, $\min_i Rc_i$) helps but does not fix it ($1.44\text{--}1.75\times$, growing with N). *RTT-equalization rules out RTT bias*: the penalty stays $\sim 1.9\times$ across a $4.5\times$ sweep of cross-flow RTTs (cross-flow RTT up to $1.8\times$ the long flow’s).

Trace diagnosis. At $N = 4$ (Figure 5 left) the long flow collapses to ~ 0.38 Gbps while the four cross flows hold ~ 23.2 Gbps each for the entire 19 ms contention window, with zero PFC and zero loss — *near-winner-take-all*: at every shared link the local single-bottleneck flow wins and the long flow is squeezed. This is an equilibrium, not a transient: with 500 MB flows ($10\times$ the contention, $\sim 7,000$ RTTs) the ratio is $1.975\times$, the long flow flat at 0.39 Gbps for 150 ms. *Nor a tuning artifact*: sweeping $\eta \in \{0.90, 0.95, 0.98\}$, R_{AI} to $25\times$ its reference (1 Gb/s), and min-rate at $N=4$ leaves the ratio at $1.74\text{--}1.97\times$, PFC 0 (`run_round5.py`); the largest R_{AI} helps most — additive recovery is directionally right but orders too slow (§III).

III. THE LIMITS OF SENDER-SIDE CONTROL

A natural first instinct — one we explored exhaustively — is to keep all state at the sender, preserving HPCC’s stateless switch. Six sender-only variants sit behind a new `mb_mode` knob (`mb_mode = 0` byte-identical to stock, verified): five heuristic changes to `UpdateRateHp`, and the most direct — a full sender-side mirror of the RCP recursion.

A. Five heuristic variants

The five heuristics attack different levers of HPCC’s update law: a k -aware additive boost scaled by the congested-hop count ($1.63\times$, the best case, at $\gamma=50$); a debiased max/mean blend of per-hop u_i ($1.95\times$ — the u_i are nearly equal here); a responsibility-weighted MD ($1.89\times$) and a k -th-root MD

($1.92\times$) — the MD branch barely fires at η -hold; and a share-aware additive increase on every flow ($1.71\times$). Stock is $1.96\times$; the target is 1.0.

B. Isolating placement: identical observations, private vs. shared state

We isolate placement in a deterministic fluid model of one link: every controller receives identical (y, q) at identical instants — same rule, gains, initialization; only the *location of the recursion state* differs (`run_consistency.py`; flow 2 arrives at $t=5$ ms). Shared register: the late arrival inherits the evolved state, 12.5/12.5 Gbps. Private, synchronized: equally fair — information suffices. Private, staggered: the newcomer initializes fresh, both multiply identically thereafter, and the ratio freezes at exactly 2:1. A sender cannot replicate the register’s *history*.

C. The full stack: a virtual-RCP mirror

The fluid model isolates the mechanism; `mb_mode 6` confirms it in the full stack: the sender runs the switch’s recursion *privately* per hop from its own INT samples of C, y, q (identical rule and gains, $R_0 = C/2$, its base RTT as interval), sending at $\min_i \hat{R}_i$. The real stack adds per-flow sampling and feedback-delay asymmetry beyond the fluid model’s arrival asymmetry — why the mirror falls short even synchronized.

The mirror reaches only $1.560\times$ synchronized and $1.654\times/0.813\times$ with the long/cross side arriving 5 ms late (direction set by arrival order), while the identical law at the switch holds $1.003\text{--}1.008\times$ throughout.

D. The structural reason: consistency, not information

Two observations explain the pattern. *First, the η -hold equilibrium*: once the contended links settle at target utilization η , HPCC’s multiplicative term $1/\max c \approx 1$ and every flow holds its current rate — whoever grabbed more at startup

keeps it. Additive nudges (modes 1, 5) are orders of magnitude too slow relative to flow lifetimes (200 ms $\approx 10^4$ RTTs, no convergence); MD-side modifications (modes 3, 4) rarely fire at the hold point.

Second — what mode 6 isolates — private explicit-rate state preserves history. The recursion’s raw inputs are sender-visible (exactly in the fluid model; sampled in the stack); what a private copy cannot reproduce is the register’s evolved history. A multiplicative update on independently initialized state preserves the ratio set at arrival forever (§III-B); the same update on one shared per-link register makes every flow — incumbent or newcomer — adopt the same R within one RTT, eliminating the history dependence. This is why RCP [5], [6] and XCP [3] place the computation in the network. The scheduling side sharpens the picture (`run_round4.py`): per-flow equal-weight DRR at every port under unchanged (windowed) HPCC leaves $1.97\times$ — the NIC-scaled window underfeeds the long flow; a scheduler cannot allocate service never offered. Removing the window lets DRR equalize ($0.997\times$) — but at ~ 0.9 Gbps per flow, $\sim 13.6\times$ every flow’s oracle FCT: rates settle near the sender floor (the η -hold’s additive path back is too slow) and the fabric idles; a fixed maxBDP window sits between ($1.52\times$). Scheduling can enforce equality; the max-min allocation — equal shares at full utilization — also requires a signal telling each sender how much to offer, and the absolute oracle-relative metric separates the outcomes ($13.6\times$ vs $1.06\times$). Per-flow queues also mean per-flow state at every port; the register needs three words. Our claim is thus scoped: for private reconstructions of an explicit fair rate, placement of the state is decisive — six sender-side designs fail while the identical law at the switch succeeds. It is not a formal impossibility for every endpoint law: contractive designs (AIMD-style convergence toward equal windows) escape the history argument in principle, though the AIMD-flavored schemes we measure (DCQCN, DCTCP) still show $1.6\text{--}2.3\times$ (§V-C).

IV. RDMA-RCP DESIGN

The empirical and structural arguments of §III motivate a design that (a) makes the *dominant* flows yield rather than merely nudging the victims, and (b) places a fair-rate *signal* at the switch, since the sender cannot recover the fair share from a load signal alone. We do not have the switch count flows and divide; instead each port runs a standard RCP control law on aggregate load and queue (below) whose fixed point is the max-min fair rate C/N . We package the most minimal such mechanism as an *operating mode* alongside stock HPCC on the same INT transport, selected per fabric or isolated slice (§IV-C).

Background: RCP The Rate Control Protocol (RCP) [5], [6] is a router-assisted congestion control in which each link advertises a *single* rate R — the rate *each* flow should send at, not an aggregate budget — to all flows crossing it, holding no per-flow state. Routers stamp R into passing packets and each flow sends at the minimum R along its path. With N flows each sending R , the controller drives $NR \rightarrow C$ (idle

Algorithm 1 RDMA-RCP switch — per egress port j

```

1: State  $R_j$  (fair rate),  $t_j$  (last update),  $B_j$  (txByte snapshot)
2: Const  $C_j$  link capacity;  $\alpha=0.4$ ,  $\beta=0.226$ ;  $d$  control interval ( $\approx$  RTT);  $R_{\text{floor}}$ 
3: procedure ONDEQUEUE( $\text{pkt}$ ,  $j$ )  $\triangleright$  every packet leaving port  $j$ 
4:   if  $R_j$  uninitialised then
5:      $R_j \leftarrow C_j/2$ ;  $t_j \leftarrow \text{now}$ ;  $B_j \leftarrow \text{txBytes}_j$   $\triangleright$  conservative start
6:   end if
7:   if  $\text{now} - t_j \geq d$  then  $\triangleright$  one control interval elapsed
8:      $y \leftarrow (\text{txBytes}_j - B_j) \cdot 8 / (\text{now} - t_j)$   $\triangleright$  observed load
9:      $q \leftarrow \text{queueBytes}_j \cdot 8$   $\triangleright$  backlog (bits)
10:     $R_j \leftarrow R_j \cdot (1 + (\alpha(C_j - y) - \beta q/d)/C_j)$   $\triangleright$  RCP update
11:     $R_j \leftarrow \text{clamp}(R_j, R_{\text{floor}}, C_j)$ 
12:     $t_j \leftarrow \text{now}$ ;  $B_j \leftarrow \text{txBytes}_j$ 
13:  end if
14:  if  $\text{pkt.fairRate} = 0$  then  $\triangleright$  first hop on the path
15:     $\text{pkt.fairRate} \leftarrow R_j$ 
16:  else
17:     $\text{pkt.fairRate} \leftarrow \min(\text{pkt.fairRate}, R_j)$   $\triangleright$  path-min
18:  end if
19:   $\text{txBytes}_j \leftarrow \text{txBytes}_j + \text{sizeof}(\text{pkt})$ 
20: end procedure

```

link $\Rightarrow R$ rises; standing queue $\Rightarrow R$ falls), so at equilibrium $R \approx C/N$ — the max-min (processor-sharing) share of the flows bottlenecked there, tracked *without counting* N : as flows arrive or leave, R self-adjusts. RCP supplies §III’s missing ingredient: a fair-share signal computed where flow count is implicitly observable — the link. RDMA-RCP runs this control law once per egress port and carries its R in the INT header; a flow’s path-minimum R approximates network-wide max-min fairness (unfairness *ratio* within 1.2% of the oracle’s; absolute per-flow FCT overheads are reported separately in §V-G), since a flow bottlenecked elsewhere contributes less load here and the loop hands the slack to the rest.

A. The mechanism

The entire protocol is Algorithms 1 and 2: one scalar fair rate per egress port, path-minimum stamped into a single header field, adopted by the sender from the returning ACK — no per-flow switch state, no per-hop record, no receiver changes.

At the switch (Alg. 1). Each egress port keeps a single scalar fair rate R_j , refreshed once per control interval d — a fabric-wide constant set to the maximum base RTT, $21.4\mu\text{s}$ in all our experiments — by the standard RCP rule $R \leftarrow R(1 + (\alpha(C - y) - \beta q/d)/C)$, where y is the load observed over the last interval and q the queue backlog. The term $\alpha(C - y)$ chases spare capacity (positive while the link is under-utilized, pushing R up) and $\beta q/d$ brakes when a standing queue forms (pulling R down); they balance at $y \approx C$, $q \approx 0$, which is

Algorithm 2 RDMA-RCP sender (receiver echoes the header unchanged)

```

1: on QP-create:  $rate \leftarrow R_{\text{start}}$   $\triangleright$  cold-start; distinct from  $R_{\text{floor}}$ 
2: procedure ONACK( $a$ )
3:   if  $a.fairRate = 0$  then return  $\triangleright$  no signal yet
4:   end if
5:    $rate \leftarrow \text{clamp}(a.fairRate, R_{\text{floor}}, NICrate)$   $\triangleright$  adopt path-min fair share
6: end procedure

```

the $R \approx C/N$ fixed point above. We use the RCP defaults $\alpha = 0.4$, $\beta = 0.226$ (untuned; the result is insensitive to them, §V-I). The port’s entire state is three numbers — R_j , the last update time, and a byte-counter snapshot — with *no* per-flow data.

On the wire. The FS INT mode (`IntHeader::FS`) carries one 64-bit field, `fairRate`; the serialized cost is 8 bytes per packet *independent of hop count* (vs. 8 bytes *per hop* for stock HPCC). Each switch min-aggregates R_j into this field (first hop initialises it), so the receiver sees the path-minimum fair rate and echoes the header back unchanged — no receiver changes.

At the sender (Alg. 2). The new ACK handler `HandleAckFs` reads `fairRate` from the returning ACK and sets the sending rate to it, clamped to $[R_{\text{floor}}, NICrate]$. That single assignment is the whole sender control loop. Two parameters are deliberately distinct: the cold-start rate R_{start} (1 Gbps; sent before the first ACK) and the adopted-rate floor R_{floor} (100 Mbps; `fs_min_rate`) — conflating them turns $N \leq C/R_{\text{start}}$ into a hard fan-in bound (measured in §V-K). ACKs are per-QP, in order, on a fixed reverse path, so the newest write wins; reordering multipath would need an epoch.

B. Startup smoothing

Two cold-start details: the switch initializes $R = C/2$ (not C , else the first stamp is line rate), and the sender starts at $R_{\text{start}} = 1$ Gbps so the first RTT does not blast before any ACK returns; adopted rates may fall to R_{floor} thereafter. The signal is path-complete in one RTT; convergence reaches 5% of fair share by ~ 0.6 ms, 1% by 2.3 ms (§V) — path-length-independent.

C. Additive integration and coexistence

RDMA-RCP is `cc_mode == 11`. The mode-3 (HPCC) and mode-10 (PINT) wire formats and switch/sender code paths are completely untouched; we add a fourth case to the INT header mode and its serialization, the switch and sender mode-dispatch, and one branch in the validation driver. The driver also forces the QP window to 0 in FS mode (rate-only control; §V-H shows why a NIC-scaled window must not remain). In the shipped `cc_mode 11` the INT wire format is a global choice — one mode per fabric. To test per-class deployment we added `cc_mode 12`: stock’s 42-byte layout (stock packets

unchanged) with per-packet class dispatch; the FS class pays stock’s full header cost — conservative for our claim. Both gates hold (all-stock $\equiv$ mode 3 exactly; all-FS $\equiv$ mode 11’s $1.005\times$, zero PFC).

Coexistence is not free (negative result). On the $N=4$ parking lot with the long flow in the RCP class and the crosses in the stock class, *neither class gets its fair share*: rate-only RCP overshoots (21.6 vs 12.5 Gbps fair, stock squeezed to 11.1; $0.51\times$), the windowed variant under-shoots (6.2 Gbps, $2.53\times$), and a separate priority queue does not help (HPCC’s queue stays near zero; it forfeits its turns) — non-compliant flows are invisible to R ’s inference exactly as flow count is invisible to a sender.

We then tested the obvious remedy: a weighted-DRR egress scheduler (byte-deficit DRR, `dwrr_weights`) reserving each class its exact 50% per-link entitlement. *It changes nothing* ($0.511\times$): the scheduler never binds, because stock HPCC’s INT still reports *port-global* bytes and queue — the stock class sees $u \approx 1$ from RCP traffic, holds low, and never offers its reserved share, which RCP then absorbs. Coexistence requires the reservation to extend into the *telemetry* — per-class C , y , q — i.e. true slicing, or a separate fabric. All variants ship with the artifact (`run_mixed_mode.py`); PFC is zero throughout.

V. EVALUATION

A. Methodology

All experiments use the deterministic parking-lot benchmark of §II, driven by our `hpcc-validation.cc`. Flows are 50 MB unless noted; sim time is 200 ms; the optimized binary is used throughout. We compare against stock HPCC (`cc_mode 3`), stock HPCC multi-rate, and HPCC-PINT. The simulator is the original HPCC ns-3 code base upgraded to a tagged ns-3.45 release — interface-level changes only; control logic and wire formats unchanged — verified by reproducing the original paper’s baseline comparisons (HPCC vs. DCQCN, TIMELY, DCTCP, and PINT across WebSearch and FB-Hadoop workloads at 30–70% load on a 320-server fat-tree). The full reproduction record ships with the artifact,³ and its verdict is that the upgraded code matches the original paper’s central findings. Metrics, used consistently below: *unfairness ratio* = long-flow FCT / mean cross-flow FCT (oracle = 1.0); *oracle-relative FCT* P_i = per-flow FCT / fluid max-min oracle FCT ($P_i < 1$ means a flow beat its oracle FCT by taking capacity the oracle assigns to others); *generalized unfairness* (heterogeneous cases) = worst-long / best-flow P_i ; *raw long/short ratio* = mean long / mean short FCT (ECMP study); *slowdown* = FCT / same-size standalone FCT; *coflow JCT* = max FCT over a coflow. “Zero PFC” statements refer to the evaluated buffer/PFC configuration (§V-J).

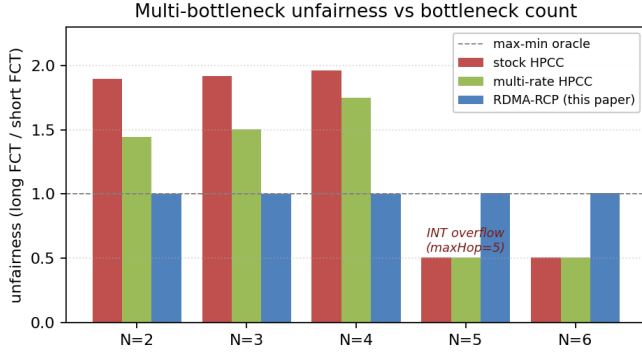


Fig. 2. Multi-bottleneck unfairness vs bottleneck count N . RDMA-RCP is flat ≈ 1.0 across the entire sweep; stock and multi-rate HPCC fail at $N \leq 4$ and break at $N \geq 5$ (INT-overflow regime).

B. Headline: N -sweep

Stock and multi-rate HPCC are starved across $N = 2-4$ (penalty $1.90-1.96\times$ for stock, $1.44-1.75\times$ for multi-rate), then break entirely at $N \geq 5$, where the per-hop INT record overflows the `maxHop` cap (the apparent $0.51\times$ there is the overflow artifact, not a real allocation). RDMA-RCP is flat at $1.003-1.007\times$ across the entire sweep — its one-field format does not grow with path length. HPCC-PINT, carrying one compressed utilization field, inherits load-based control and still suffers $\sim 1.8\times$ ($1.77-1.88\times$) — the fix requires a fair-share signal, not a smaller INT footprint. All RDMA-RCP runs have zero PFC events.

C. The failure spans our evaluated stack

Is multi-bottleneck unfairness HPCC-specific? We ran every scheme in our stack on the same benchmark with harness parity (window-based schemes keep their window; DCQCN, TIMELY, DCTCP run windowless per their reference configurations — DCTCP as a datacenter-TCP baseline, not an RDMA scheme). Table I: every scheme *in our evaluated stack* is unfair to the multi-bottleneck flow and worsens with bottleneck count — DCQCN worst at $N=4$ ($2.32\times$). HPCC is the norm here, not an outlier; only the RCP mode reaches max-min, and only HPCC and the RCP mode hold zero PFC. Untested controllers (PowerTCP, Bolt, On-Ramp, Swift) are outside the claim; a same-benchmark comparison is future work.

D. When max-min is the objective: ring-collective coflow JCT

Collective communication motivates multi-bottleneck fairness: a step completes only when its *slowest* flow completes (JCT = max FCT), and the slowest flows are precisely the cross-pod, multi-bottleneck ones stock HPCC starves. We run a 16-flow ring coflow — one communication phase of a ring allreduce (50 MB per host to its ring successor; 4 inter-pod, 4

³Reproduction record and methodology: <https://github.com/UNOCourseDemo/hpcc-fs/tree/main/examples/hpcc/repro-verification>. The raw per-run results (554 MB compressed) are archived at https://1drv.ms/f/c/6052297178cce52b/IgAsSQ7gNfdhQrNejod27CpkAYQs8xZs4QtGc_KKVVWZjQLs.

TABLE I
UNFAIRNESS RATIO AND PFC PAUSES ON THE PARKING LOT: EVERY *evaluated* SCHEME FAILS MULTI-BOTTLENECK MAX-MIN; ALL WORSEN WITH N .

scheme	$N=2$	$N=3$	$N=4$	PFC
DCQCN	$1.70\times$	$1.90\times$	$2.32\times$	6–34
TIMELY	$1.83\times$	$1.96\times$	$2.01\times$	72–156
DCTCP	$1.61\times$	$1.72\times$	$1.78\times$	256–280
HPCC	$1.90\times$	$1.92\times$	$1.96\times$	0
RDMA-RCP	$1.003\times$	$1.003\times$	$1.005\times$	0

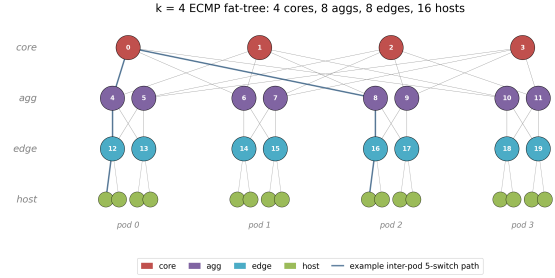


Fig. 3. The $k = 4$ ECMP fat-tree (4 cores, 8 aggs, 8 edges, 16 hosts in 4 pods); the highlighted line is a representative inter-pod 5-switch path. The tree-fabric topology (unique shortest paths, no ECMP) is described in the text.

intra-pod, 8 same-edge) over all hosts of the $k=4$ fat-tree, across five ECMP seeds. A full allreduce chains $2(N-1)$ such phases; if later phases see similar contention and no compute overlap, their *absolute* savings accumulate — we do not simulate the multi-phase collective. RDMA-RCP cuts JCT by 19.5–22.7% (mean 21.4%) at *every* seed, and makes it nearly placement-invariant (17.9–18.0 ms across seeds, vs. stock’s 22.3–23.3). Eight added 2 MB background flows leave the gain intact (+20.6%) and themselves speed up $\sim 3\times$ (0.9–1.3 vs 2.2–4.3 ms) — a short flow’s fate under near-winner-take-all is a placement lottery; max-min removes it. A header-size control: padding the RCP class to stock’s 42-byte layout (§IV-C, all-FS) still gives 19.0 vs 23.3 ms — 18% under byte-identical headers (23% at 8 bytes). The gain is fairness, not header thinness.

E. Structurally different topologies

Tree fabric (3-tier, no ECMP). A 3-tier tree with unique shortest paths (15 nodes; 25 Gbps inter-switch, 100 Gbps host; Figure 3). Stock penalty $1.36\times$, RDMA-RCP **$1.003\times$** , PFC = 0.

$k = 4$ fat-tree (with ECMP). The canonical fat-tree (20 switches + 16 hosts; Figure 3), hash-based ECMP; 4 inter-pod (5-switch) plus 4 intra-pod (3-switch) flows. On the *oracle-normalized penalty* (long-path vs. short-path slowdown, each against the max-min oracle, removing placement effects): stock $1.34\times$, RDMA-RCP **$1.001\times$** . The *raw* long/short ratio stays $\sim 1.32\times$ under RDMA-RCP — not CC unfairness but placement: ECMP hashes some flows onto under-loaded cores, inflating the raw ratio for any controller. PFC = 0 for both; path-min aggregation is path-invariant, so ECMP needs no design

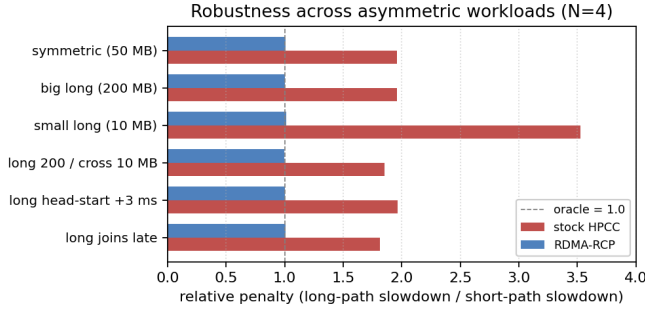


Fig. 4. Robustness across asymmetric-size and staggered-start variants at $N = 4$. RDMA-RCP is within 1.0 ± 0.012 across all six scenarios, including the small-long worst case where stock reaches $3.53\times$.

changes. *Hash-seed sensitivity*. Because a single deterministic run fixes one placement, we re-ran the workload across *twenty* ECMP hash seeds (an additive `ecmp_seed_offset` knob shifts every switch’s hash seed; the default preserves stock behavior), reporting the mean-long/mean-short raw ratio. Under stock HPCC that ratio varies $0.96\text{--}1.40\times$ across seeds (mean 1.14) with *no change to the congestion controller* — direct evidence that the raw spread is placement-dominated. Under RDMA-RCP the mean is 1.04 and the median 1.001 : fifteen of twenty seeds sit within 3% of parity, and the five residuals ($1.16\text{--}1.33\times$) are placements whose hash collides the long flows onto shared cores, where equal FCTs are not the oracle allocation either. At the default seed the contended case drops $1.338\times \rightarrow 1.001\times$ (seed-robust; `run_ecmp_seeds.py`).

F. Robustness across asymmetric workloads

Across six perturbations (Figure 4) RDMA-RCP holds within 1.0 ± 0.012 ; the worst stock case ($3.53\times$, small long flow) drops to $1.012\times$.

Start-time jitter. Jittering all five starts by $U[0, 1]$ ms (five seeds): stock $1.96\text{--}1.99\times$, RDMA-RCP $1.008\text{--}1.017\times$. On the $N=1$ control the pairwise spread persists at $1.10\text{--}1.39\times$ across $10\ \mu\text{s}\text{--}5\ \text{ms}$ offsets, direction flipping with arrival order — the η -hold lottery, not synchronization — while RDMA-RCP equalizes at every offset.

G. Heterogeneous and dynamic contention

The uniform one-competitor parking lot is deliberately minimal, so we stress its assumptions (Table II; `run_stress_matrix.py`): heterogeneous capacities, unequal competitor counts, two overlapping multi-bottleneck flows, and cross-flow churn, each flow scored against the event-driven progressive-filling oracle on the actual topology and schedule. A ratio alone can mask a uniform miss, so the table reports both *generalized unfairness* (worst-long / best-flow P_i ; reduces to the headline ratio in the uniform case) and the absolute P_i range. Stock HPCC’s unfairness grows with contention richness — worst with two overlapping long flows ($2.87\times$) and churn ($2.23\times$), its P_i scattered $0.41\text{--}1.34$ (some flows racing ahead of their oracle share while the longs lag) — while RDMA-RCP holds $1.001\text{--}1.007\times$

TABLE II
BEYOND THE UNIFORM PARKING LOT: GENERALIZED UNFAIRNESS AND THE ABSOLUTE PER-FLOW RANGE OF ORACLE-RELATIVE FCT P_i . RDMA-RCP HOLDS $\approx 1.0\times$ WITH EVERY FLOW WITHIN 5–13% OF ITS ORACLE FCT; PFC = 0 THROUGHOUT.

scenario	stock HPCC		RDMA-RCP	
	unf.	P_i range	unf.	P_i range
hetero cap. [25,50,25,100] G	$1.82\times$	$0.64\text{--}1.16$	$1.003\times$	$1.06\text{--}1.06$
unequal comp. [3,1,2,1]	$1.41\times$	$0.84\text{--}1.18$	$1.001\times$	$1.06\text{--}1.06$
two overlapping longs	$2.87\times$	$0.41\text{--}1.19$	$1.007\times$	$1.05\text{--}1.06$
cross-flow churn	$2.23\times$	$0.60\text{--}1.34$	$1.002\times$	$1.06\text{--}1.13$

TABLE III
WINDOW ABLATION: THE NIC-SCALED WINDOW BREAKS FAIRNESS; A FAIR-RATE-SCALED CANONICAL RCP WINDOW PRESERVES IT (ALL ZERO PFC).

N	rate-only (default)	NIC-scaled window	$cwnd = R \cdot \text{RTT}$
2	$1.003\times$	$1.50\times$	$1.004\times$
3	$1.003\times$	$2.00\times$	$1.004\times$
4	$1.005\times$	$2.49\times$	$1.006\times$

with $P_i \in [1.05, 1.13]$: *every* flow lands within 5–13% of its own oracle FCT — the wire calibration alone is 5.8%, so the measured convergence residual is ≈ 0 outside churn (1.13 worst case). Mean bottleneck utilization is *higher* than stock throughout ($0.65\text{--}0.78$ vs $0.51\text{--}0.70$). PFC = 0 everywhere; $4\text{--}8\times$ RTT spreads applied to the mode itself leave the ratio at $1.003\text{--}1.008\times$. Jointly stressing the mode — churn plus a $4\times$ RTT spread plus d misconfigured $4\times$ too small, at once — leaves the ratio at $1.000\times$ while the absolute P_i rise to $1.21\text{--}1.39$: the compound stress costs uniform slowdown, which the absolute metric exposes and the ratio alone would hide (`run_round3.py`). (INT admits only rates $\{25, 50, 100, 200, 400\}$ Gbps; heterogeneity stays within that set.)

H. Ablation: windows must scale with the fair rate

RDMA-RCP defaults to rate-only operation — the per-flow window cap is disabled (§IV). Table III: re-enabling HPCC’s `var_win` degrades the penalty to $1.5\text{--}2.5\times$, worsening with path length — that window is sized from the NIC line rate, not the multi-hop bottleneck. The problem is the window’s *scale*, not windowing: $cwnd = R \cdot \text{baseRTT}$ (`fs_rcp_window`) matches rate-only fairness while bounding in-flight data — a cap must track R , not the NIC.

I. Parameter sensitivity

We use the standard RCP defaults ($\alpha = 0.4$, $\beta = 0.226$) untouched. Sweeping each parameter one at a time at $N = 4$ — $\alpha \in [0.2, 0.6]$, $\beta \in [0.1, 0.4]$, $\text{startup } R_0/C \in [0.25, 1.0]$, $\text{minRate} \in [100, 2000]$ Mb/s — the penalty stays within $[1.004, 1.008]\times$ with zero PFC throughout (even $R_0 = C$: the cold-start absorbs the first interval); the parameters are not load-bearing in this family (§V-G adds a joint stress). The interval d tolerates misconfiguration asymmetrically: $2\text{--}4\times$

too large or $2\times$ too small stays $1.003\text{--}1.011\times$; $4\times$ too small degrades to $1.212\times$ (updates outrun feedback) — still far below every baseline, PFC zero.

J. Queueing and PFC

For context, every run uses the validated HPCC buffer model: 4 MB shared buffer with dynamic PFC thresholds, ECN marking $k_{\min}/k_{\max}/p_{\max} = 100\text{ KB}/400\text{ KB}/0.2$ at 25 Gbps (400/1600 KB at 100 Gbps), 1000 B payload, $1\text{--}2\mu\text{s}$ link delays. With the smoothed startup (§IV-B), RDMA-RCP produces **zero PFC events** on every workload in the N -sweep and the robustness matrix. ECN marking remains; queue length is bounded (peak $\approx 46\text{ KB}$ at $N = 4$, versus $\approx 118\text{ KB}$ for stock HPCC on the same workload); no flow loss. Do-no-harm holds throughout: the RDMA-RCP additions (implementation name HPCC-FS, `cc_mode 11`) only add code, and stock `cc_mode 3` smoke tests and N -sweep outputs are byte-identical to the pre-change baseline.

K. High fan-in and idle-port staleness

Incast fan-in, and the rate floor (`run_round4/5/6.py`): S senders to one 25 Gbps receiver link. Fan-in exposes why R_{start} and R_{floor} must not be conflated: with the floor tied to the 1 Gbps cold-start (our initial implementation), $S \leq C/R_{\text{floor}} = 25$ is a *hard feasibility bound* — at $S=64$ every sender is clamped above its 0.39 Gbps fair share and PFC enforces the sharing the endpoint cannot adopt (2,045 pauses, 142 ms cumulative). Decoupling the floor (100 Mbps) removes the overload: PFC = 0 through $S=128$. What remains is a genuine *throughput* limitation, and it is not a startup transient: sweeping flow size at $S=64$, the mode’s lifetime utilization (bound/FCT) is 0.47 at 200 KB and *falls* to 0.25 at 50 MB, versus stock HPCC’s 0.92 at every size (64 pauses, 36.5 ms paused) — the port oscillates between multiplicative overshoot and the floor rail, and a lower floor deepens the collapse (0.26 at $S=128$, 50 Mbps floor). The decoupled floor also *moves* rather than removes the bound: $S \leq C/R_{\text{floor}} = 250$ at 100 Mbps; at $S=256$ senders pin at the floor (util 0.99 only because floor $\approx$ fair share); beyond, structural overload. A receiver-side rate trace confirms the oscillation directly: aggregate goodput swings 3–25 Gbps with 91% of bins below $C/2$ (`run_round6.py`). The damping knob exists: interval $\times 2$ (`fs_d_scale`) restores 0.84–0.88 at $S=64$ (both sizes, PFC-free); $\times 4$ reaches 0.87–0.94 (44 pauses at $S=64$, none at $S=128$) — with the $N=4$ headline unchanged ($1.003\times$, PFC 0; cf. §V-I). A residual gap to stock HPCC’s 0.92 remains at the default interval; adaptive interval selection is future work. The floor never binds when fair shares exceed it; every fairness result reproduces exactly. *Idle-port staleness*: R updates only on dequeue, so an idle port freezes its last R . Congest a port with four flows, idle it $g \in \{0.1, 1, 10, 100\}$ ms, release a 1 MB probe: FCT 0.373 ms at *every* gap — faster than a fresh port’s 0.713 ms ($R_0=C/2$ ramp); the drain tail’s last dequeues restore R to $\approx C$. The frozen value is benign after *naturally draining* flows — the tested trajectory; an abrupt halt (failure,

reroute) could freeze a low R , which we do not model; idle aging (reset after $k \cdot d$ without dequeues) would cover it.

L. HPCC’s home turf: competitive FCT, lower queue

What does the RCP mode cost on *single-bottleneck* workloads — HPCC’s home turf? We measure the validated incast benchmark (3 senders $\rightarrow$ 1 receiver over 25 Gbps; 12 flows, 100–400 KB), sweeping the sender’s startup rate. Two things stand out: RDMA-RCP holds far less queue than HPCC at every startup (38–64 KB vs 143 KB), and HPCC’s only lead — mean FCT at the conservative 1 Gbps startup (297.5 vs 258.6 μs , $\sim 15\%$) — closes and reverses at 8 Gbps (242.0 vs 258.6 μs at one-quarter the queue, zero PFC). The 8 Gbps point leaves the multi-bottleneck headline undisturbed ($1.003\text{--}1.004\times$, PFC = 0); the headline runs use 1 Gbps. On these workloads the fairness mode costs little-to-no FCT and holds less queue. Before the first fair-rate ACK a sender emits at most $\text{startup} \times \text{baseRTT}$ ($\approx 2.7\text{ KB}$ at 1 Gbps), and the $\text{cwnd} = R \cdot \text{RTT}$ variant (§V-H) restores a hard per-flow cap thereafter; the fan-in limit of this startup bound is measured directly in §V-K. RDMA-RCP still lacks HPCC’s per-hop, window-bounded reaction — §V-K measures the $S=64$ boundary — and recovering it while keeping fairness is the hybrid’s target (§VII).

M. Per-flow rate trace, before vs after

The before-vs-after rate trace (Figure 5) is the cleanest qualitative result: collapse to ~ 0.4 Gbps under stock, lock-in at fair share under RDMA-RCP.

N. Scope of the headline claim

The $\approx 1.0\times$ headline is a statement about the *congestion-control component* of multi-bottleneck unfairness on contention benchmarks — not a claim of universal FCT improvement on production workloads. Six first-class limits. (i) *Mixed short-flow workloads* do not exhibit the penalty we target: under a WebSearch distribution stock HPCC’s oracle-normalized penalty is already $1.001\times$ (RDMA-RCP: $0.581\times$ — short flows run *ahead* of their max-min share: policy deviation, not superiority), and on mean slowdown RDMA-RCP trades $\sim 15\%$ off multi-bottleneck flows for $\sim 45\%$ on short paths. Short-flow-FCT workloads [7], [8] may prefer stock HPCC; max-min matters when multi-bottleneck flows gate a job (§V-D). Per-service selection needs per-class link or telemetry isolation (§IV-C: an unsliced port serves neither class); the deployment unit is a fabric or slice dedicated to max-min traffic, not a global replacement. (ii) *Fabric scale*: our $k = 6, 8$ scaling workload does not form persistent inter-pod contention (both penalties < 1), so we added a *saturating* $k=6$ case — 108 identical 20 MB inter-pod flows, every stage at capacity under ECMP (`run_round4.py`): coflow makespan $42.9 \rightarrow 36.7$ ms (-14% ; -11% with the INT header padded to stock size), Jain $0.910 \rightarrow 0.924$, PFC = 0 at 45 switches. Both schemes traverse *identical* ECMP placements (same hash, seeds, 5-tuples), so the placement penalty ($2.9\times$ vs $3.4\times$ the 12.8 ms perfect-balance fluid bound) is common, so the

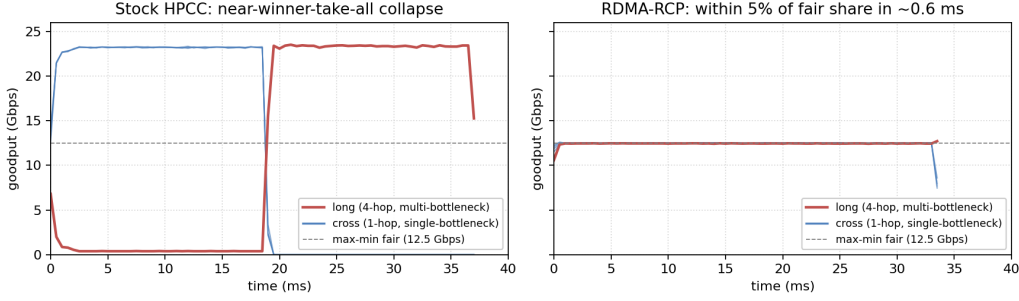


Fig. 5. Per-flow goodput at $N = 4$, stock HPCC (left) vs RDMA-RCP (right). Stock collapses the long flow to ~ 0.4 Gbps while cross flows hold ~ 23 Gbps for 18 ms (near-winner-take-all). RDMA-RCP converges all five flows to within 5% of the max-min fair share (12.5 Gbps) in ~ 0.6 ms and to within 1% by ~ 2.3 ms.

delta is attributable to the controller in aggregate (startup, queueing, utilization, fairness) — a scale-and-performance sanity check, *not* a max-min validation; a path-resolved oracle needs path instrumentation ($k=8$: future work). (iii) *ECMP placement*: a raw $\sim 1.3\times$ long/short spread from hash placement remains; RDMA-RCP removes the CC component only. (iv) *Coexistence*: RDMA-RCP is a fabric-wide mode — unsliced sharing fails even under DRR (§IV-C). (v) *High fan-in*: at $S \geq 64$ the rate mode sustains only 25–50% utilization at the default control interval (0.84–0.94 with a 2–4 $\times$ interval, §V-K), and $S \leq C/R_{\text{floor}}$ is a hard bound; combined with (iv), fan-in-heavy services still favor the stock side of the slice boundary. (vi) *Fairness unit*: max-min here is per QP/flow — measured: a job opening two QPs obtains exactly $2.00\times$ a one-QP job’s bandwidth on a shared bottleneck (`run_round5.py`); job-level fairness would need weighted rates ($R_i = w_i R$) or QP-count policing, which we do not evaluate.

VI. RELATED WORK

Datacenter transport. DCTCP [9], DCQCN [10], TIMELY [11], and Swift [12] trade buffering for latency via ECN- and delay-based control; Homa [7] and pFabric [8] prioritize short flows; HPCC [1] drives control from per-hop INT, with On-Ramp [13] and PowerTCP [14] extending the precise-signal philosophy. Across this line the target is single-bottleneck FCT, tail latency, or queueing; §V-C shows every member we evaluate fails multi-bottleneck fairness.

In-network telemetry. INT [15] lets the data plane stamp per-hop state into packets on programmable pipelines, and HPCC was among the first to drive congestion control directly from it. The cost is header overhead, which later work minimizes; HPCC-PINT [16] compresses the per-hop record into one probabilistic field; Poseidon [17] restructures INT control for commodity deployability. RDMA-RCP continues this minimal-telemetry direction but changes *what* the field carries: an explicit fair *rate*, min-aggregated along the path — one 64-bit field, like PINT, yet a decision rather than a measurement.

Explicit-rate and switch-assisted control. Switch-computed rates are classic: ATM ABR’s ERICA [18] computed per-port fair shares decades ago; XCP [3] split efficiency from fairness;

RCP [6] reduced it to one packet-stamped fair-rate scalar per link, senders taking the path minimum. RCP is the closest mechanism to ours: RDMA-RCP adopts its update rule but situates it as a *minimal fairness signal injected into an existing INT load-signaling system* rather than a standalone protocol, keeping HPCC’s single-rate sender and per-hop transport. Prior RCP work established explicit-rate fairness in general fabrics; what had not been measured is whether deployed INT-based RDMA stacks exhibit the multi-bottleneck gap, and whether a telemetry-carried fair rate closes it inside such a stack — this paper’s contribution. RDMA-RCP’s per-port update and path-min aggregation *are* RCP’s; the endpoint differs (rate-only by default; the canonical window performs equally, Table III), the control interval is a fabric constant, updates are event-driven, and the port applies $R_0 = C/2$ and a floor. Over deploying RCP as a separate protocol, RDMA-RCP adds packaging, not control: one field on the INT transport already running, a byte-identical stock path, per-mode selection — no second stack.

In-switch fairness. Fair queueing [19], DRR [20], CSFQ [21], and AFD [22] enforce fairness in the scheduler; §III measures the boundary — scheduling equalizes only what senders offer, and without a rate signal the equal outcome is far from the max-min allocation. RDMA-RCP instead *advertises* one fair rate per port — no per-flow tracking, no extra queues — implementable on the match-action pipelines [23] INT already assumes.

Closely related switch-assisted DC schemes. Bolt [24] accelerates HPCC’s loop with sub-RTT per-flow switch assistance, targeting tail latency and ramp-up — a different objective: its feedback carries load/supply, not a fair share; its multi-bottleneck behavior is unknown (head-to-head: future work).

VII. DISCUSSION

Hybrid path. RDMA-RCP inherits RCP’s fair, low-queue character without HPCC’s single-flow precision (§V-L); a future hybrid caps HPCC at the fair rate, $\min(\text{rate}_{\text{HPCC}}, R)$, with a fair-rate-scaled window (§V-H).

Scaling and hardware. The saturating $k=6$ case is in §V-N; the non-saturating $k = 6, 8$ workloads form no persistent

contention (penalties <1 , 0 PFC, no added overhead). On hardware we offer a mapping, not a prototype: three registers per port (R , timestamp, byte snapshot); a dequeue-triggered update ($1/C$, β/d precomputed; two multiplies, two subtractions, a clamp); a 64-bit compare-and-write path-min per packet. A target providing an atomic per-port stateful update could serialize the interval test and fair-rate update; whether the state and multiplies fit one target's stateful stage is *unverified*, as is dequeue-time queue-depth access. We did take the step short of hardware: a *fixed-point simulation* (`fs_fixed_point`) — Q16 multiplier, precomputed $1/C$ and $\beta/(dC)$ reciprocals, an 8-entry elapsed-interval LUT, no divider — reproduces the N -sweep ($1.001\text{--}1.004\times$ vs $1.003\text{--}1.005\times$), ring JCT ($+0.9\%$), and damped fan-in (0.81 vs 0.84 ; `run_round6.py`). One hazard surfaced: an R register in integer $Mbps$ deadlocks recovery from the rate floor (growth truncates to zero below ~ 2.5 Mbps; the fan-in case pins at the sender floor, 0.27) — Kbps units in the same 32-bit register remove it. Register *units*, not arithmetic width, were the binding choice. A Tofino prototype remains future work.

Limitations. Three topology families; in-sim baselines only (DCQCN, TIMELY, DCTCP, multi-rate HPCC, PINT) — no Bolt/PowerTCP, no testbed; the in-order per-QP ACK assumption (§IV) holds in the evaluated stack but reordering multipath would need an epoch field; per-class coexistence requires link or telemetry isolation we do not design (§IV-C); the ECMP study is a small controlled workload (seed-swept), not a realistic mixed-size combination. All numbers come from one simulator — but its control logic is the original HPCC implementation with interface-level modernization only, verified end-to-end by reproducing the original baselines, and the $N \leq 4$ regime sits within the 5-hop INT budget by construction; a belt-and-braces run on the unmodified legacy build remains future work.

VIII. CONCLUSION

Multi-bottleneck unfairness spans every scheme in our evaluated stack, and HPCC's instance is a stable, RTT-independent, near-winner-take-all equilibrium ($\sim 2\times$; $3.5\times$ for small flows; unchanged over $10\times$ longer backlogs). Six sender-side corrections fail — including an exact private mirror of RCP — and per-flow fair queueing equalizes without approaching the allocation ($1.97\times$ windowed; $\sim 13\times$ the oracle FCT windowless); a controlled fluid experiment shows why: with identical observations, law, and epochs, private explicit-rate state still freezes arrival-order asymmetry; a shared register erases it. RDMA-RCP carries that register's value in one INT field — three words of mutable state per port — reaching $1.005\times$ — flows within $5\text{--}13\%$ of oracle FCT in the heterogeneous matrix ($21\text{--}39\%$ under compound stress), PFC-free in the parking-lot, heterogeneous, coflow, and scale experiments — rate-only or canonically windowed, converging sub-millisecond, path-length-independent, stock HPCC byte-for-byte unchanged. Coexistence on a shared link fails even under a DRR reservation (§IV-C): the mode belongs on sliced telemetry or a separate fabric.

REFERENCES

- [1] Y. Li, R. Miao, H. H. Liu, Y. Zhuang, F. Feng, L. Tang, Z. Cao, M. Zhang, F. Kelly, M. Alizadeh, and M. Yu, "HPCC: High precision congestion control," in *Proc. ACM SIGCOMM*, 2019, pp. 44–58.
- [2] D. Bertsekas and R. Gallager, *Data Networks*, 2nd ed., 1992.
- [3] D. Katabi, M. Handley, and C. Rohrs, "Internet congestion control for future high bandwidth-delay product environments," in *Proc. ACM SIGCOMM*, 2002, pp. 89–102.
- [4] Alibaba, "HPCC reference simulator (ns-3)," <https://github.com/alibaba-edu/High-Precision-Congestion-Control>, 2019, `int-header.h: static const uint32_t maxHop = 5`.
- [5] N. Dukkapi and N. McKeown, "Why flow-completion time is the right metric for congestion control," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 36, no. 1, pp. 59–62, 2006.
- [6] N. Dukkapi, "Rate Control Protocol (RCP): Congestion control to make flows complete quickly," Ph.D. dissertation, Stanford University, 2008.
- [7] B. Montazeri, Y. Li, M. Alizadeh, and J. Ousterhout, "Homa: A receiver-driven low-latency transport protocol using network priorities," in *Proc. ACM SIGCOMM*, 2018.
- [8] M. Alizadeh, S. Yang, M. Sharif, S. Katti, N. McKeown, B. Prabhakar, and S. Shenker, "pFabric: Minimal near-optimal datacenter transport," in *Proc. ACM SIGCOMM*, 2013.
- [9] M. Alizadeh, A. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, and M. Sridharan, "Data center TCP (DCTCP)," in *Proc. ACM SIGCOMM*, 2010, pp. 63–74.
- [10] Y. Zhu, H. Eran, D. Firestone, C. Guo, M. Lipshteyn, Y. Liron, J. Padhye, S. Raindel, M. H. Yahia, and M. Zhang, "Congestion control for large-scale RDMA deployments," in *Proc. ACM SIGCOMM*, 2015, pp. 523–536.
- [11] R. Mittal, V. T. Lam, N. Dukkapi, E. Blem, H. Wassel, M. Ghobadi, A. Vahdat, Y. Wang, D. Wetherall, and D. Zats, "TIMELY: RTT-based congestion control for the datacenter," in *Proc. ACM SIGCOMM*, 2015, pp. 537–550.
- [12] G. Kumar, N. Dukkapi, K. Jang, H. M. G. Wassel, X. Wu, B. Montazeri, Y. Wang, K. Springborn, C. Alfeld, M. Ryan, D. Wetherall, and A. Vahdat, "Swift: Delay is simple and effective for congestion control in the datacenter," in *Proc. ACM SIGCOMM*, 2020.
- [13] S. Liu, A. Ghalayini, M. Alizadeh, B. Prabhakar, M. Rosenblum, and A. Sivaraman, "Breaking the transience-equilibrium nexus: A new approach to datacenter packet transport," in *Proc. USENIX NSDI*, 2021.
- [14] V. Addanki, M. Apostolaki, M. Ghobadi, S. Schmid, and L. Vanbever, "PowerTCP: Pushing the performance limits of datacenter networks," in *Proc. USENIX NSDI*, 2022, pp. 51–70.
- [15] C. Kim, A. Sivaraman, N. Katta, A. Bas, A. Dixit, and L. J. Wobker, "In-band network telemetry via programmable dataplanes," in *Proc. ACM SIGCOMM (Industrial Demo)*, 2015.
- [16] L. S. Yang, R. Ben Basat, M. Mitzenmacher, M. Apostolaki, A. Klein, E. Saito, and A. Mendelson, "PINT: Probabilistic in-band network telemetry," in *Proc. ACM SIGCOMM*, 2020, pp. 662–677.
- [17] W. Wang, M. Moshref, Y. Li, G. Kumar, T. S. E. Ng, N. Cardwell, and N. Dukkapi, "Poseidon: Efficient, robust, and practical datacenter CC via deployable INT," in *Proc. USENIX NSDI*, 2023.
- [18] S. Kalyanaraman, R. Jain, S. Fahmy, R. Goyal, and B. Vandalore, "The ERICA switch algorithm for ABR traffic management in ATM networks," *IEEE/ACM Transactions on Networking*, vol. 8, no. 1, 2000.
- [19] A. Demers, S. Keshav, and S. Shenker, "Analysis and simulation of a fair queueing algorithm," in *Proc. ACM SIGCOMM*, 1989.
- [20] M. Shreedhar and G. Varghese, "Efficient fair queueing using deficit round robin," in *Proc. ACM SIGCOMM*, 1995.
- [21] I. Stoica, S. Shenker, and H. Zhang, "Core-stateless fair queueing: Achieving approximately fair bandwidth allocations in high speed networks," in *Proc. ACM SIGCOMM*, 1998.
- [22] R. Pan, L. Breslau, B. Prabhakar, and S. Shenker, "Approximate fairness through differential dropping," *ACM SIGCOMM Computer Communication Review*, vol. 33, no. 2, 2003.
- [23] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, 2014.
- [24] S. Arslan, Y. Li, G. Kumar, and N. Dukkapi, "Bolt: Sub-RTT congestion control for ultra-low latency," in *Proc. USENIX NSDI*, 2023.